

PONTIFICIA UNIVERSIDAD JAVERIANA

Facultad de Ingeniería

Introducción a la Inteligencia Artificial

Taller EDA – Fase 1

Análisis Exploratorio y Preprocesamiento

Dataset: Adult (Census Income) – UCI Repository

Docente:

Ing. Julio Omar Palacio Niño, M.Sc.

palacio_julio@javeriana.edu.co

Objetivo:

Predecir si una persona gana más o menos de 50.000 USD al año.

Bogotá D.C., 2026

Contents

1	Carga y unión de los datos	2
2	Análisis gráfico (EDA)	2
2.1	Distribución de la variable objetivo	2
2.2	Boxplots de variables numéricas	2
2.3	Variables categóricas vs income	7
2.4	Matriz de correlación	8
3	Preprocesamiento	9
3.1	Orden del preprocesamiento y por qué importa	9
3.2	Marcado de valores faltantes	10
3.3	Codificación de la variable objetivo	11
3.4	Dummificación	11
3.5	Partición 80/20 — punto de separación definitivo	11
3.6	Imputación de nulos — fit en train, apply en test	12
3.7	Winsorización de outliers — fit en train, apply en test	12
3.8	Normalización — fit en train, apply en test	13
3.9	Dummificación	13
3.10	Normalización	14
4	Resumen del flujo	16

1 Carga y unión de los datos

El dataset viene partido en dos archivos: `adult.data` y `adult.test`. Dado que el enunciado pide trabajar con el conjunto completo, se unieron con `pd.concat`. El dataset corresponde al censo de ingresos de Estados Unidos de 1994, publicado en el repositorio UCI por Becker y Kohavi [1].

Al cargarlos aparecieron dos problemas puntuales. El primero: `adult.test` trae una línea de texto basura al inicio (`| 1x3 Cross validator`), por lo que hay que saltársela con `skiprows=1`. El segundo: en ese mismo archivo, la columna `income` tiene un punto al final (`<=50K.`, `>50K.`) que no está en `adult.data`. Si no se quita, pandas los lee como categorías distintas y termina habiendo cuatro clases en vez de dos, lo cual arruinaría todo el análisis posterior.

Después de unir y limpiar esos detalles, el dataset queda con **48.842 filas y 15 columnas**.

2 Análisis gráfico (EDA)

Este documento explica las decisiones que se tomaron en el notebook, paso por paso. No es un resumen del código sino una justificación de por qué cada cosa se hizo como se hizo.

2.1 Distribución de la variable objetivo

Lo primero que se revisa es `income`, la variable que se quiere predecir. El resultado muestra un desbalance claro: el 76% de los registros corresponde a personas que ganan `<=50K`, y solo el 24% a quienes ganan `>50K`. Esto importa porque, si no se tiene en cuenta al partir el dataset ni al elegir métricas, un modelo podría simplemente predecir siempre `<=50K` y tener 76% de *accuracy* sin haber aprendido nada útil — fenómeno bien documentado en la literatura sobre clasificación con clases desbalanceadas [2].

2.2 Boxplots de variables numéricas

Los boxplots muestran cómo se distribuye cada variable numérica y permiten identificar valores que se alejan del comportamiento general. A continuación se describe lo que revela

cada uno.

age

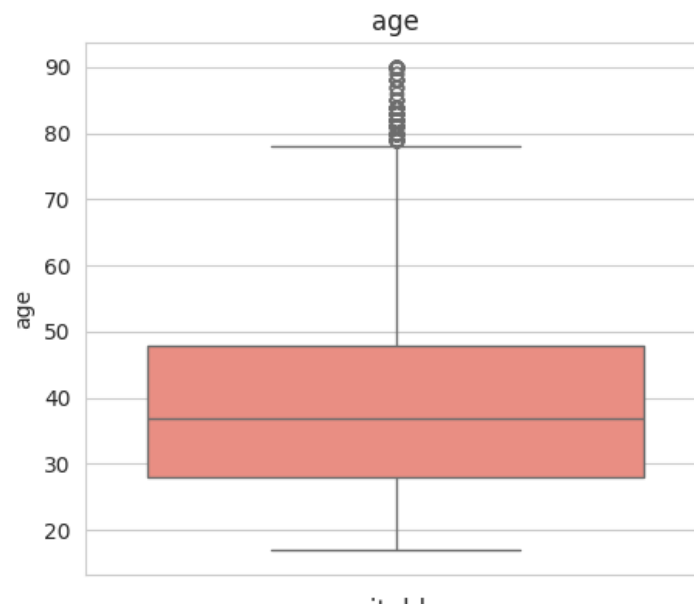


Figure 1: Distribución de la edad en el dataset.

La edad se concentra entre los 28 y los 48 años, con una mediana alrededor de los 37. Eso indica que la mayor parte del dataset corresponde a adultos en edad laboral activa. Por encima de los 80 años aparecen varios puntos sueltos que se distancian del resto, lo cual es una señal que vale la pena registrar para el preprocesamiento.

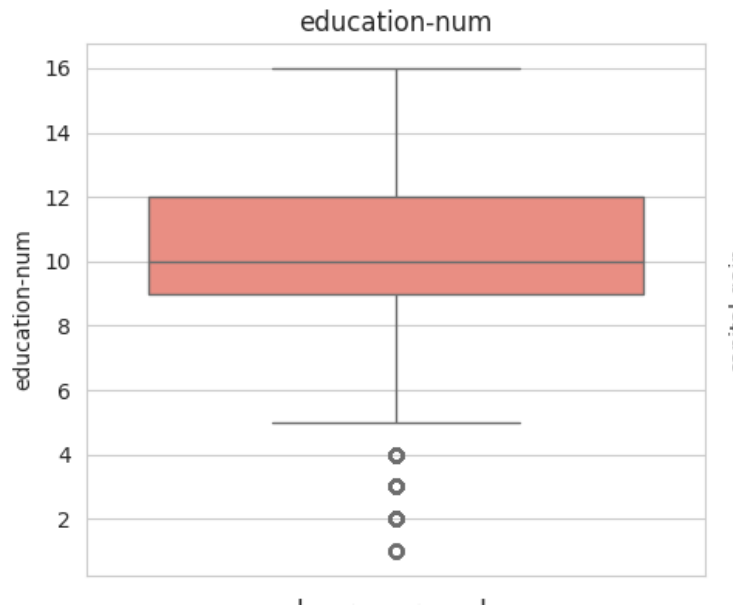
education-num

Figure 2: Distribución del nivel educativo codificado numéricamente.

Esta variable codifica el nivel educativo en una escala del 1 al 16. El 50% central va del nivel 9 al 12, que corresponde aproximadamente a bachillerato y algo de universidad. Por debajo del nivel 5 aparecen algunos puntos aislados que representan personas con muy poca escolaridad. Son valores atípicos según el gráfico, aunque no necesariamente incorrectos.

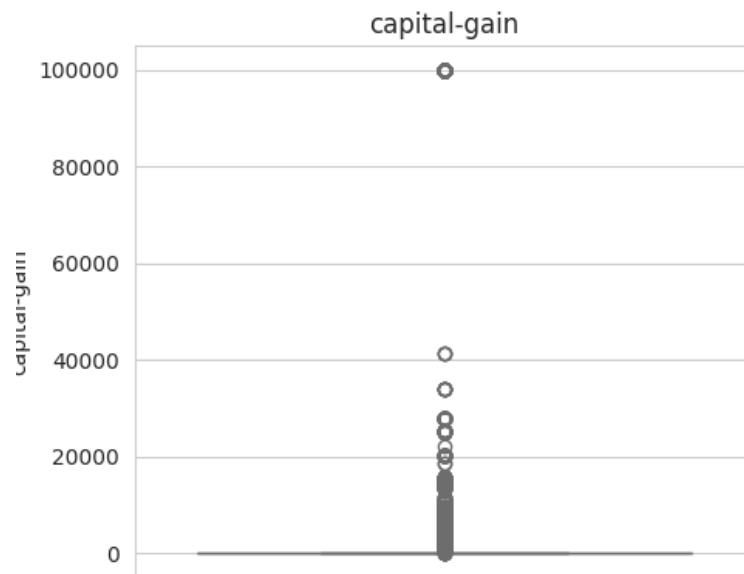
capital-gain

Figure 3: Distribución de las ganancias de capital.

La caja está completamente aplastada en cero: la gran mayoría de personas no reportó ninguna ganancia de capital. Los puntos dispersos por encima corresponden a quienes sí invirtieron y obtuvieron ganancias reales. La distribución es prácticamente binaria: o no hay ganancia, o hay una cifra considerable.

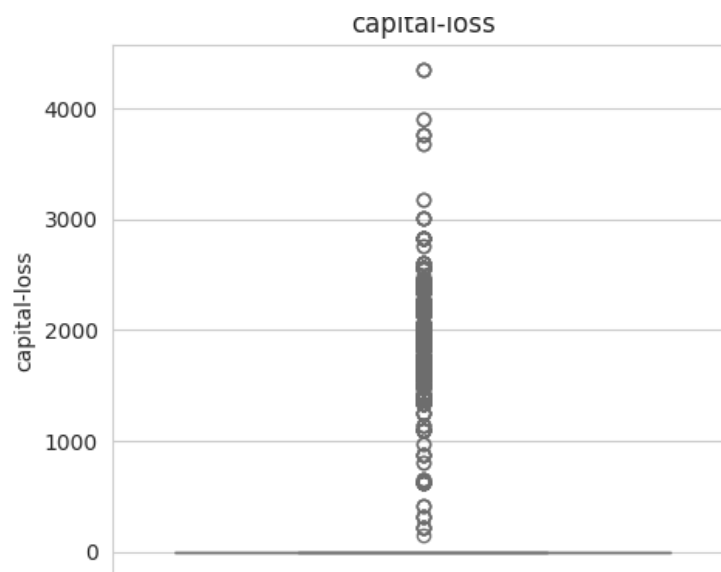
capital-loss

Figure 4: Distribución de las pérdidas de capital.

El patrón es el mismo que en `capital-gain`: casi todo el dataset está en cero y solo una minoría reporta pérdidas. Los puntos que se alejan hacia arriba reflejan una realidad económica concreta, no errores en los datos.

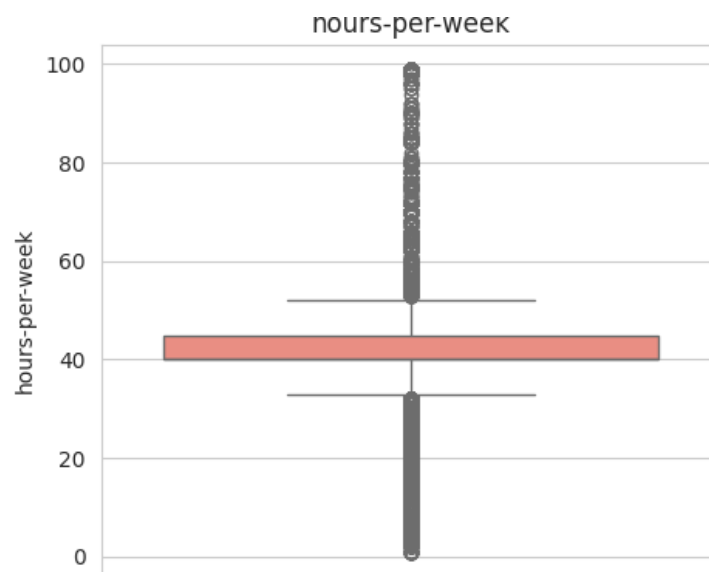
hours-per-week

Figure 5: Distribución de las horas trabajadas por semana.

La jornada de 40 horas domina con claridad: la caja es casi plana alrededor de ese valor. Aun así, hay casos en ambos extremos, con personas que reportan desde 1 hasta 99 horas semanales. Esa amplitud hace que la variable tenga un rango considerablemente mayor al que describe la mayoría de la población, algo que tendrá que atenderse en el preprocesamiento.

2.3 Variables categóricas vs income

Al cruzar las categóricas con `income` aparecen relaciones interesantes. Por ejemplo, a mayor nivel educativo, mayor probabilidad de pertenecer al grupo `>50K`. De igual manera, las personas casadas (`Married-civ-spouse`) tienen una proporción notablemente más alta en ese grupo. A continuación se muestra el cruce de `sex` vs `income`:

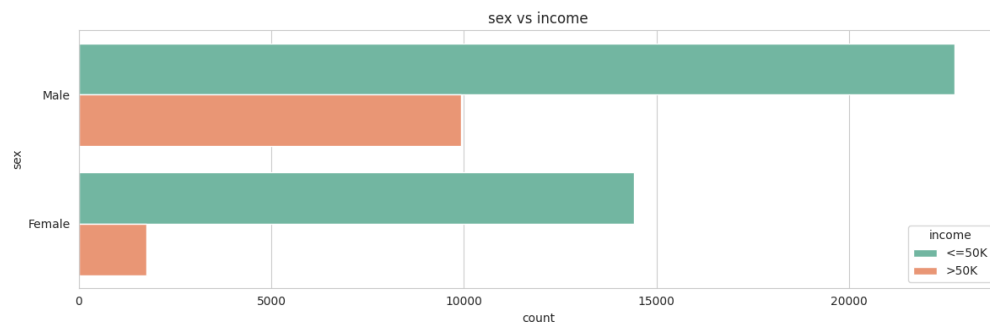


Figure 6: Distribución del ingreso según el sexo reportado en el censo.

La gráfica muestra una diferencia notable: los hombres tienen una proporción de >50K considerablemente mayor que las mujeres. En términos absolutos, hay alrededor de 10.000 hombres en el grupo >50K frente a poco más de 1.700 mujeres. Esto no significa que el modelo deba reproducir ese sesgo, sino que refleja las condiciones socioeconómicas del censo de 1994 en el que se basa el dataset.

2.4 Matriz de correlación

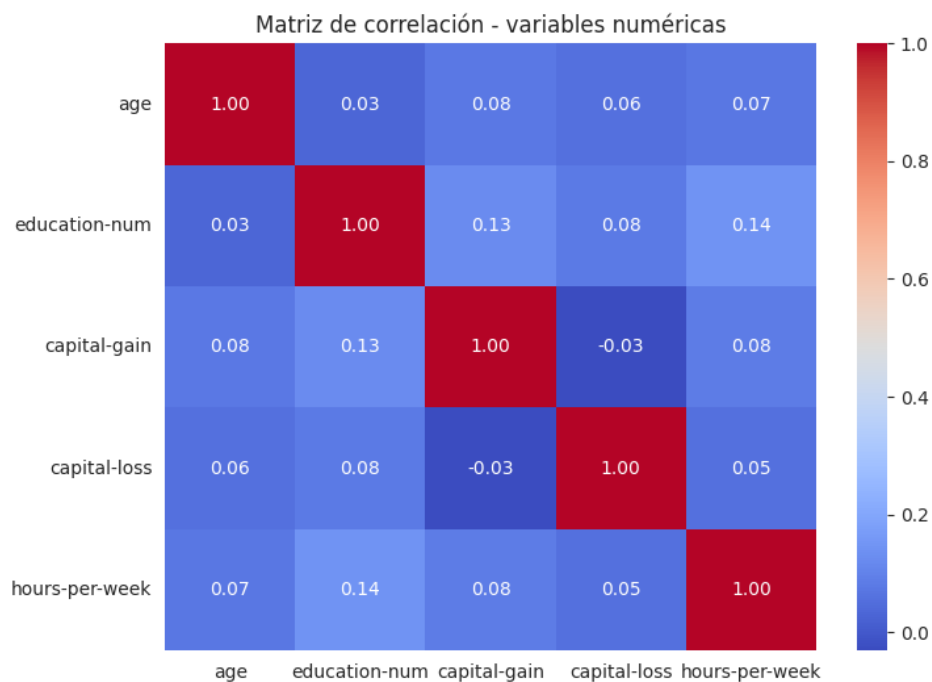


Figure 7: Matriz de correlación entre las variables numéricas del dataset.

Las correlaciones entre variables numéricas son bajas en general, lo cual es positivo: significa que ningún par de variables está midiendo exactamente lo mismo. Los valores más altos son entre `education-num` y `capital-gain` (0.13), y entre `education-num` y `hours-per-week` (0.14), lo que tiene sentido: a mayor nivel educativo, mayor probabilidad de tener un trabajo de más horas y de invertir capital. En todo caso, ninguna correlación supera 0.15, puesto que no hay riesgo real de redundancia entre las variables numéricas.

3 Preprocesamiento

3.1 Orden del preprocesamiento y por qué importa

Antes de describir cada paso, es fundamental explicar el orden en que se ejecutaron, ya que el orden en sí es una decisión técnica con consecuencias directas sobre la validez de los resultados.

El principio que gobierna todo el flujo es la **separación aprendizaje–predicción**: ningún parámetro que el modelo vaya a usar (medias, modas, límites de outliers, columnas de dummies) debe calcularse con información del conjunto de pruebas [3]. Cuando eso ocurre, se produce *data leakage*: el conjunto de pruebas “contamina” el entrenamiento, haciendo que el modelo parezca funcionar mejor de lo que realmente funcionará sobre datos nuevos.

La tabla 1 resume el orden adoptado y la justificación de cada decisión:

Table 1: Orden del preprocesamiento y justificación de cada decisión.

Paso	Sobre qué datos	Por qué
Marcar ? como NaN	Dataset completo	Solo cambia representación, no aprende nada
EDA gráfico	Dataset completo	Exploración visual, no modifica datos
Codificar <code>income</code> (0/1)	Dataset completo	Es la variable objetivo; no introduce fuga
Dummificación	Dataset completo	Garantiza columnas idénticas en train y test
SPLIT 80/20	—	Punto de separación definitivo
Imputar nulos	Fit: train → Apply: test	La moda se aprende solo del train
Winsorizar outliers	Fit: train → Apply: test	Los límites IQR se calculan solo del train
Normalizar	Fit: train → Apply: test	Media y std se aprenden solo del train

La dummificación es la única excepción al principio general de hacer todo después del split. El motivo es técnico: si se aplica por separado en train y test, una categoría que aparezca solo en uno de los dos generaría conjuntos de columnas distintos, haciendo que el modelo falle al predecir. Aplicarla sobre el dataset completo antes del split garantiza que ambos subconjuntos tengan exactamente las mismas columnas. Este es un compromiso reconocido y justificado en la práctica del preprocesamiento [4].

3.2 Marcado de valores faltantes

Este dataset tiene un detalle poco común: los valores faltantes no vienen como NaN sino como el carácter ?. En consecuencia, si se ejecuta `df.isnull()` directamente, pandas reporta cero nulos, aunque sí los hay. Por eso, el primer paso fue reemplazar ? por NaN con `df.replace('?', np.nan)`. Este paso solo cambia la representación: no imputa, no calcula ningún parámetro ni modifica ningún valor real.

Las columnas afectadas son tres: `workclass`, `occupation` y `native-country`, con nulos en alrededor del 7% de las filas.

3.3 Codificación de la variable objetivo

`income` se transformó a 0 ($\leq 50K$) y 1 ($> 50K$) antes del `split`. Esto no introduce fuga porque `income` es la variable que se quiere predecir, no una variable predictora: el modelo nunca va a “ver” el `income` del test durante el entrenamiento, independientemente de cuándo se codifique.

3.4 Dummificación

Antes de dummificar, se eliminó la columna `education` ya que `education-num` contiene exactamente la misma información pero en números (un valor del 1 al 16 por cada nivel educativo). Mantener las dos sería redundancia pura.

Para el resto de las variables categóricas se aplicó *one-hot encoding* con `pd.get_dummies`. La opción más inmediata habría sido asignar un número a cada categoría (`Private = 1`, `Self-emp = 2`, etc.), pero eso introduciría relaciones de orden y distancia que no existen: el modelo interpretaría que `Government (3)` es “más” que `Private (1)`. Kuhn y Johnson [5] describen este problema explícitamente: la codificación ordinal en variables nominales introduce relaciones artificiales que los modelos interpretan como información real. El *one-hot encoding* evita ese problema representando cada categoría como algo independiente.

El argumento `drop_first=True` elimina una columna por cada variable para evitar **multicolinealidad perfecta**: si `sex_Male` es 0, ya se sabe que la persona es `Female`, y la segunda columna sobra.

Como resultado, el dataset pasa de 14 columnas predictoras a alrededor de 90, principalmente por `native-country`.

3.5 Partición 80/20 — punto de separación definitivo

Se eligió una división **80/20**: 80% para entrenamiento (~39.000 filas) y 20% para pruebas (~9.700 filas). Con casi 50.000 registros en total, dejar 9.700 filas para test es suficiente para obtener una estimación confiable del rendimiento [4].

Se usó `stratify=y` para asegurarse de que la proporción 76/24 se mantenga en ambos subconjuntos. Sin estratificación, el test podría quedar con una distribución muy distinta por puro azar, haciendo que las métricas sean engañosas. Japkowicz y Stephen [2] demuestran

que el desbalance de clases tiene un impacto directo y medible sobre la evaluación, lo que justifica preservarlo explícitamente.

A partir de este punto, `X_test` y `y_test` quedan completamente aislados. Todo lo que sigue se ajusta **solo** con los datos de entrenamiento.

3.6 Imputación de nulos — fit en train, apply en test

La imputación se hace después del split porque la moda es un parámetro estadístico que se aprende de los datos. Si se calculara sobre el dataset completo, la moda del test contaminaría la del train.

El procedimiento correcto es calcular la moda de cada columna afectada **solo con `X_train`**, y luego usar esa misma moda para rellenar los nulos en `X_test` sin recalcularla [6].

Ante eso hay dos caminos: eliminar las filas con nulos o imputarlas. Eliminar no es válido aquí por dos razones. La primera es numérica: el 7% de 48.842 filas son más de 3.400 registros que se perderían junto con toda su información. La segunda es más sutil: los nulos no son aleatorios. La mayoría de los ? en `workclass` y `occupation` corresponden a personas mayores o sin empleo formal; eliminarlos sesgaría el dataset hacia una subpoblación no representativa.

La moda es válida porque estas variables son categóricas: la media y la mediana no aplican para texto. Además, en `workclass` y `native-country` la categoría dominante ya representa más del 70% de los registros, así que rellenar con ella no distorsiona la distribución de forma perceptible.

3.7 Winsorización de outliers — fit en train, apply en test

En la sección de boxplots se identificaron valores extremos en varias variables. Para tratarlos se usó el **método IQR** de Tukey [7]: un valor es atípico si cae fuera de $[Q1 - 1.5 \times IQR, Q3 + 1.5 \times IQR]$. Al igual que con la imputación, los límites se calculan **solo con `X_train`** y luego se aplican a `X_test`.

`capital-gain` y `capital-loss` no se modificaron: como casi todos sus valores son cero, el IQR es cero y cualquier cifra positiva queda marcada como atípica, pero esas cifras son ganancias y pérdidas reales de capital, no errores. `fnlwgt` tampoco se tocó por ser un

peso censal con extremos estadísticamente válidos.

`age` y `hours-per-week` sí se winsorizaron. Hay registros con 90 años o 99 horas semanales que distorsionan el aprendizaje. Eliminar esas filas no sería válido porque tienen información completa y correcta en las demás columnas. La winsorización (`clip`) recorta esos valores al límite del IQR sin eliminar ninguna fila, conservando toda la información y reduciendo el peso de los casos más extremos.

3.8 Normalización — fit en train, apply en test

Se usó `StandardScaler` (media 0, desviación estándar 1). El problema que resuelve es de escala: `fnlwgt` tiene valores en cientos de miles y `age` en decenas. Sin normalización, modelos como regresión logística, KNN o SVM darían más peso a `fnlwgt` simplemente por su magnitud, no por su información.

Se eligió `StandardScaler` sobre `MinMaxScaler` porque las distribuciones tienen colas y no son uniformes; `StandardScaler` es más robusto en esos casos.

El `fit_transform` se aplica **solo a `X_train`**: aprende la media y desviación estándar de ese subconjunto. El `transform` se aplica a `X_test` usando esos mismos parámetros sin recalcularlos. Si se normalizara primero el dataset completo, el scaler incluiría la distribución del test en su cálculo, introduciendo *data leakage* [3, 8]. Las pequeñas diferencias entre las estadísticas del train y del test normalizado son esperadas y correctas: confirman que el test no influyó en los parámetros del scaler.

3.9 Dummificación

Antes de dummificar, se eliminó la columna `education` ya que `education-num` contiene exactamente la misma información pero en números (un valor del 1 al 16 por cada nivel educativo). Mantener las dos sería como tener la misma columna dos veces.

Para convertir las variables categóricas restantes a formato numérico, la opción más inmediata sería asignarles un número a cada categoría: `Private` = 1, `Self-emp` = 2, `Government` = 3, y así sucesivamente. Sin embargo, eso introduciría un problema serio: el modelo interpretaría que `Government` (3) es “más” que `Private` (1), o que la distancia entre las categorías 2 y 3 es la misma que entre 5 y 6. Ninguna de esas relaciones existe en la realidad — `workclass` no tiene orden ni jerarquía, las categorías simplemente son distintas

entre sí. Kuhn y Johnson [5] describen este problema explícitamente: la codificación ordinal directa en variables nominales introduce relaciones artificiales que los modelos interpretan como información real, degradando su rendimiento.

La excepción ya se manejó antes: `education-num` sí usa números directamente porque el nivel educativo *tiene* un orden real y significativo. Un valor de 13 (Bachillerato universitario) genuinamente representa más años de escolaridad que un 9 (Bachillerato secundario). Ahí el número tiene sentido matemático; en `workclass` u `occupation`, no.

Por eso se aplicó *one-hot encoding* con `pd.get_dummies`. En vez de dejar una columna `sex` con los valores `Male` y `Female`, se crea una columna nueva por cada categoría posible, con un 1 si esa fila pertenece a ella y un 0 si no. De esa forma el modelo ve cada categoría como algo independiente, sin inventar ninguna relación de orden ni distancia entre ellas. La tabla 2 ilustra el proceso:

Table 2: Ejemplo de one-hot encoding sobre la variable `sex`.

<code>sex</code>	<code>sex_Male</code>	<code>sex_Female</code>
Male	1	0
Female	0	1
Male	1	0

Ahí entra el argumento `drop_first=True`, que elimina una de esas dos columnas nuevas. ¿Por qué? Porque son redundantes entre sí: si `sex_Male` es 0, automáticamente se sabe que la persona es `Female`, sin necesidad de tener la segunda columna. Dejarlas las dos le estaría diciendo al modelo la misma cosa dos veces, lo que puede generar problemas matemáticos en algunos algoritmos. A ese fenómeno se le llama **multicolinealidad perfecta**.

Como resultado, el dataset pasa de 14 columnas predictoras a alrededor de 90. El salto se debe principalmente a `native-country`, que tiene decenas de países distintos, cada uno con su propia columna binaria.

3.10 Normalización

Se usó `StandardScaler`, que transforma cada variable numérica para que tenga media 0 y desviación estándar 1.

El problema que resuelve es de escala: `fnlwgt` tiene valores en cientos de miles, mientras que `age` está en decenas. Sin normalización, modelos como regresión logística, KNN o SVM le darían más peso a `fnlwgt` simplemente porque sus números son más grandes, no porque sea más informativa.

Se eligió `StandardScaler` por encima de `MinMaxScaler` ya que las distribuciones no son uniformes y tienen colas. `StandardScaler` maneja mejor esos casos. Para modelos de árbol de decisión esto no cambia nada, pero para modelos basados en distancia o gradiente sí es relevante.

Orden de aplicación. Es fundamental normalizar *después* de partir el dataset, no antes. Si se aplica `fit_transform` sobre el dataset completo, el `StandardScaler` calcula la media y desviación estándar usando también las filas que luego van al conjunto de test. Eso hace que el modelo reciba, de forma indirecta, información sobre datos que se supone que nunca ha visto — un problema conocido como **data leakage** o fuga de información [3, 8].

Kaufman et al. [3] catalogan el *data leakage* como uno de los diez errores más graves en minería de datos, precisamente porque produce estimaciones de rendimiento demasiado optimistas: el modelo parece funcionar bien en evaluación pero falla en producción. La documentación oficial de scikit-learn [8] es explícita al respecto: “*Always split the data into train and test subsets first, particularly before any preprocessing steps*”.

Por eso el procedimiento correcto es:

1. Partir el dataset (`train_test_split`).
2. Ajustar el scaler **solo** con el train (`fit_transform`): aprende media y desviación de ese subconjunto únicamente.
3. Aplicar ese mismo scaler al test **sin reajustarlo** (`transform`): usa los parámetros del train, sin acceder a la distribución del test.

Las pequeñas diferencias entre las estadísticas del train y del test normalizado son esperadas y correctas: reflejan que el scaler no vio el test. Lo incorrecto sería que ambos tuvieran media exactamente 0, lo cual solo ocurre si se normalizó sobre el dataset completo.

4 Resumen del flujo

A modo de cierre, el proceso completo siguió este orden:

1. Cargar `adult.data` y `adult.test`, corregir el formato del test y unirlos.
2. Marcar `?` como `NaN` (solo representación, sin imputar).
3. Explorar gráficamente distribuciones, relaciones con `income` y correlaciones.
4. Codificar `income` como 0/1 (variable objetivo, sin riesgo de fuga).
5. Aplicar *one-hot encoding* al dataset completo (excepción justificada para garantizar columnas idénticas).
6. **Partir en 80/20 con estratificación** — punto de separación definitivo.
7. Imputar nulos con la moda calculada **solo del train**, aplicada luego al test.
8. Winsorizar `age` y `hours-per-week` con límites IQR calculados **solo del train**.
9. Normalizar con `StandardScaler`: `fit_transform` en `train`, `transform` en `test`.

El orden de los pasos 6 al 9 no es intercambiable. Cualquier paso de preprocesamiento que calcule un parámetro estadístico (moda, límites IQR, media, desviación estándar) debe ejecutarse después del split y sobre el train únicamente [3]. El test existe exclusivamente para evaluar — nunca para enseñar.

Referencias

- [1] Barry Becker **and** Ronny Kohavi. *Adult (Census Income) Dataset*. UCI Machine Learning Repository. Consultado en mayo de 2025. 1996. URL: <https://archive.ics.uci.edu/ml/datasets/adult>.
- [2] Nathalie Japkowicz **and** Shaju Stephen. “The class imbalance problem: A systematic study”. **in** *Intelligent Data Analysis*: 6.5 (2002), **pages** 429–449.
- [3] Shachar Kaufman, Saharon Rosset, Claudia Perlich **and** Ori Stitelman. “Leakage in data mining: Formulation, detection, and avoidance”. **in** *ACM Transactions on Knowledge Discovery from Data*: 6.4 (december 2012), **pages** 1–21. DOI: 10.1145/2382577.2382579. URL: <https://doi.org/10.1145/2382577.2382579>.
- [4] Aurélien Géron. *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*. 3 **edition**. O’Reilly Media, 2022. ISBN: 978-1-098-12597-4.
- [5] Max Kuhn **and** Kjell Johnson. *Feature Engineering and Selection: A Practical Approach for Predictive Models*. Chapman **and** Hall/CRC, 2019. ISBN: 978-1-138-07922-9.
- [6] Salvador García, Julián Luengo **and** Francisco Herrera. *Data Preprocessing in Data Mining*. Intelligent Systems Reference Library. Springer, 2015. ISBN: 978-3-319-10246-7. DOI: 10.1007/978-3-319-10247-4.
- [7] John W. Tukey. *Exploratory Data Analysis*. Addison-Wesley, 1977. ISBN: 978-0-201-07616-5.
- [8] scikit-learn developers. *Common pitfalls and recommended practices*. Documentación oficial de scikit-learn. Consultado en mayo de 2025. 2024. URL: https://scikit-learn.org/stable/common_pitfalls.html.